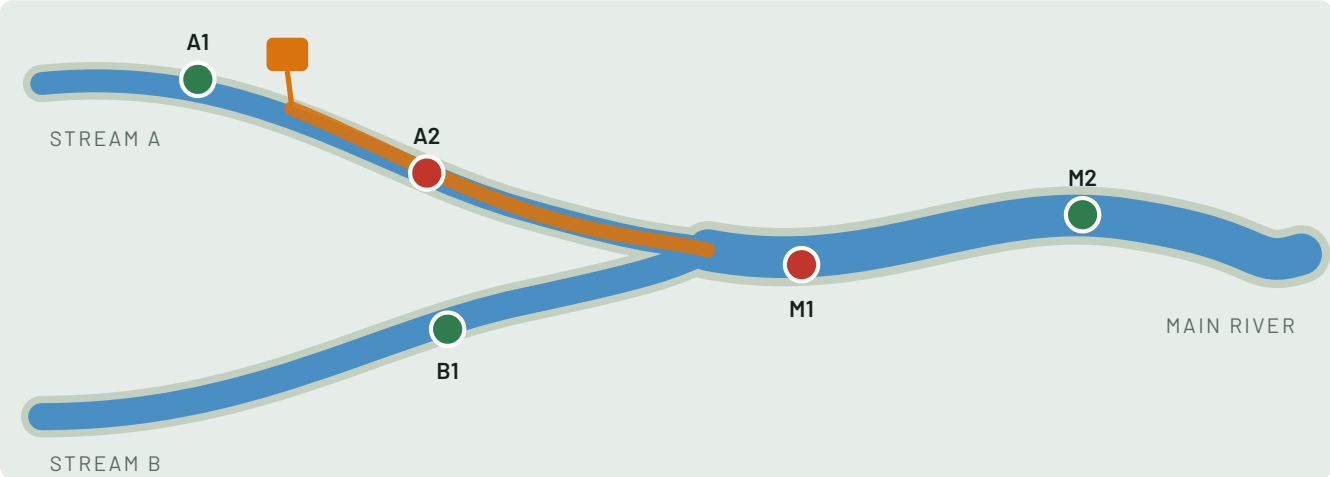
River Monitoring Node: How the Code Works

What each file does, how a sensor reading turns into a pollution alert, and how the nodes along a river work together.



A mine on stream A (orange) is picked up at A2, then at M1 below where the streams join. A1 above the mine and B1 on the other stream stay normal, so the system can say the source is between A1 and A2.

CONTENTS

1 What the system does
2 The big picture: from sensor to alert
3 The files, one by one
4 Turning voltages into readings
5 Learning what "normal" is
6 Sensor fault checks
7 The pollution rules
8 From rules to alerts
9 Comparing nodes along the river
10 Reading the serial output
11 Water speed and phone alerts
12 Settings, building and uploading
13 The other folders
14 Limits and things to know
15 Words used in this guide

SECTION 1

What the system does

Each monitoring node is an ESP32 board with four water sensors sitting in a river. It measures the water every 2 seconds, learns what the water is normally like at that spot, and raises an alert when the readings change in a pattern that matches a known kind of pollution.

The four sensors are:

SENSOR	MEASURES	UNIT	WHY IT MATTERS
Logo-Rnaenaor V2.0 pH module	How acidic or alkaline the water is	pH (0–14)	Acid mine water, alkaline cement waste and chemical spills all shift the pH.
STJF TDS Meter V1.0	Total dissolved solids (salts and minerals)	mg/L	Almost every kind of waste adds dissolved material. Rain dilutes it.
TS-300B turbidity sensor (0–4.5 V output)	How cloudy the water is	NTU	Mud, sewage and runoff make water cloudy.
DS18B20 thermometer	Water temperature	°C	Warm outflows (thermal pollution), and the TDS reading has to be corrected for temperature.

One node on its own can say *"the water here has changed, and the change looks like sewage"*. Several nodes along a river can say more: *"it isn't at the node upstream, so the source is between those two nodes"*, or *"every node sees it at once, so it's rain, not a leak"*.

> **What it is not**
> The labels are likely matches, not lab results. Four sensors can't identify a chemical. The code says which kinds of pollution fit the readings and lists the substances usually involved, so someone knows what to test a water sample for.

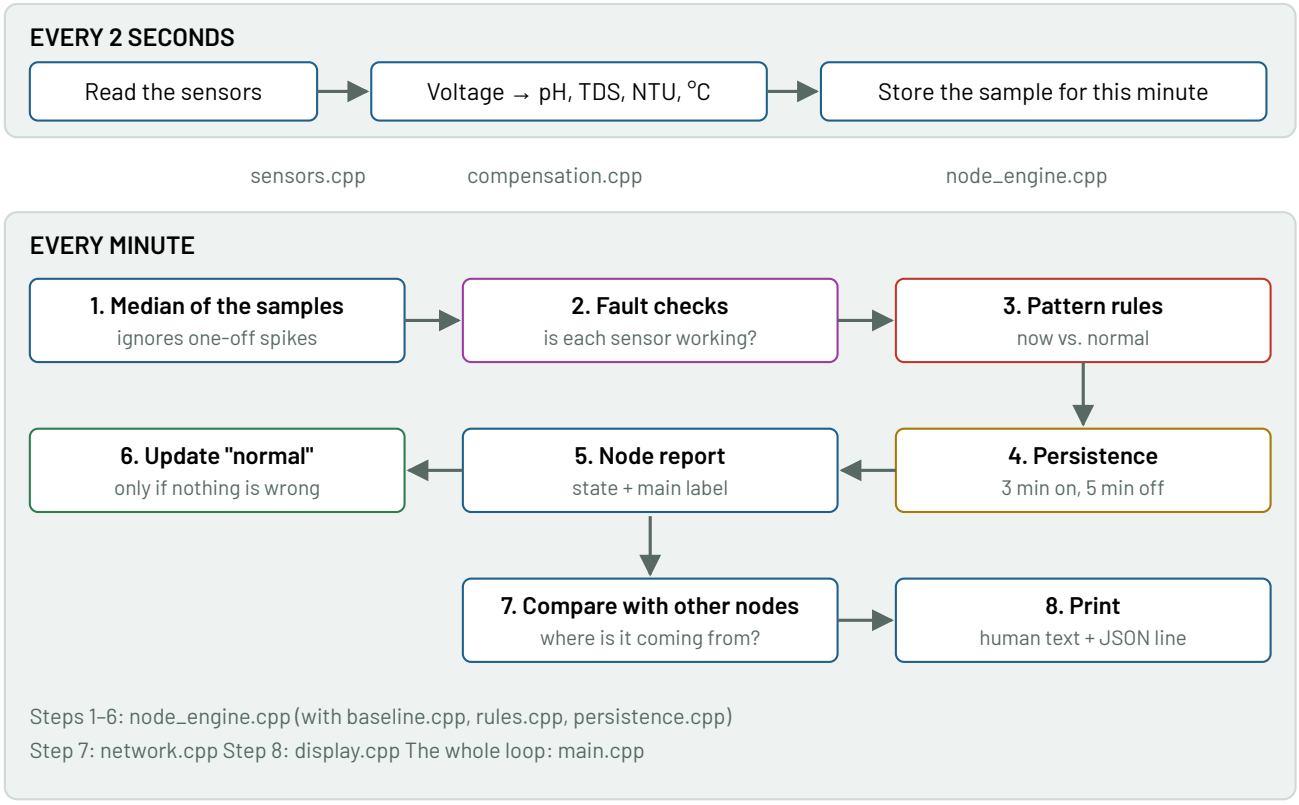
The labels it can give

GROUP	LABELS	NODE STATE
Pollution	Likely heavy metals / acid mine drainage · Likely industrial discharge · Likely alkaline waste · Likely sewage	ALERT
Watch	Likely nutrients / algae · Thermal pollution · Likely sediment · Likely salt · Likely treated wastewater or fertiliser runoff	WATCH
Filter	Likely rain / stormwater event (stops rain being mistaken for pollution)	NORMAL
Fault	A sensor is unplugged, stuck, reading impossible values, or drifting	FAULT
Learning	The node hasn't learned what normal is yet	BASELINE_BUILDING

SECTION 2

The big picture: from sensor to alert

The program has two clocks. Every **2 seconds** it takes a sample. Every **minute** it works out what that minute's water was like and decides what it means.



The order of work in `loop()` in `main.cpp`. Steps 1–6 happen inside one function, `engine_close_minute()`.

Why work per minute instead of per sample? One sample can be wrong (a bubble on the probe, electrical noise). Taking the **median** of a minute's samples (the middle value once sorted) throws those one-off spikes away. The rules then compare this minute's medians with the node's learned **baseline** ("normal" for this spot), not with fixed numbers. A TDS of 400 mg/L is normal in some rivers and a red flag in others.

The node's whole minute summary is one struct, `struct Node_report` (in `types.h`). It holds the state, main label, confidence, readings now and normal, any faults, and any other labels that are also on. It's what gets printed, and what a node would send to its neighbours over a radio link.

SECTION 3

The files, one by one

All the code for the board is in `src/`. Each `.cpp` file has a matching `.h` file listing its functions (the function prototypes). Inside each file, the functions are grouped under banners like `/////// BAROMETER ///////`, the same layout as the model rocket avionics project.

Start here

`main.cpp`	The program. `setup()` sets up the river layout, the engine and the sensors, then prints a banner. `loop()` checks for typed commands, takes a sample every 2 s, and closes the minute every minute (see section 2). `setup_river_layout()` at the bottom lists the nodes on the river.
`config.h`	Every setting and threshold in one place: modes, pins, calibration values, how long to learn "normal", and every rule's numbers. Nothing in the rules is hard-coded, so tuning means changing this file only.

Shared building blocks

`types.h / .cpp`	The enums (sensors, labels, categories, fault codes, node states) and structs shared by every file (`Reading`, `Rule_result`, `Node_report`), plus small helpers like `sensor_name()` and `state_name()`.
`labels.h / .cpp`	The words that go with each label: its code (e.g. `LIKELY_SEWAGE`), long and short names, category, the pollutants it might be, and what it means. The rules only return a label number. `get_label_info()` turns that number into text.

Hardware

`sensors.h / .cpp`	Talks to the real sensors. `wake_up_sensors()` sets up the ADC and the DS18B20. `read_sensors()` reads all four (16 ADC reads averaged per sensor) and returns a `struct Reading`. `last_sensor_voltages()` is for calibrating.
`compensation.h / .cpp`	The maths that turns voltages into pH, TDS (corrected to 25 °C) and NTU, and checks whether a temperature reading is real (section 4). It has no Arduino code in it, so it can be tested on a PC.

The brain of one node

`node_engine.h / .cpp`	Ties the minute together. `engine_add_sample()` stores each 2 s sample. `engine_close_minute()` takes the medians, checks faults, runs the rules, updates persistence, builds the report and updates the baseline. The same engine runs the real node, the simulated nodes, the PC tests and the web simulator.
`baseline.h / .cpp`	Learns and remembers "normal" for each sensor at this spot (section 5). Also keeps the day's highest and lowest pH (for the algae rule) and watches for the pH probe slowly drifting.
`rules.h / .cpp`	The sensor fault checks (section 6) and the nine pattern rules plus the rain filter (section 7). Each rule is a short function that scores required and optional conditions.
`persistence.h / .cpp`	Stops alerts flickering: a label only turns on after 3 minutes in a row and only turns off after 5 minutes without it (section 8).

The river network

`network.h / .cpp`	Keeps every node on the river in an array: its branch (stream), distance down the river, and latest report. Branches can flow into each other (two streams joining a main river). `network_assess()` compares this node with the others to work out where pollution is coming from (section 9).
`water_speed.h / .cpp`	Works out how fast the water is moving from pollution reaching one node and then another further down (section 11).

Output and control

`display.h / .cpp`	Everything printed to the serial monitor. The rules and the engine never print. Prints the minute block for people to read, a JSON line for computers, ALERT ON/OFF lines, and the network list. Colours can be switched off with `USE_COLOUR`.
`commands.h / .cpp`	Reads commands typed into the serial monitor: `list`, `add <id> <metres> [place]`, `del <id>`, `move <id> <metres>`, `cal` (prints sensor voltages for calibrating), `arm` / `disarm` (sensor-problem notifications), `test` (send a test notification), `wifi` (WiFi status) and `help`.
`report_text.h / .cpp`	Turns codes into words (fault reasons, the network finding sentence, label codes), so the serial monitor and the phone say the same thing.
`phone_messages.h / .cpp`	Builds the messages sent to the phone: one line of JSON text each, for the status, the node list, alerts and water speeds (section 11).
`phone_alerts.h / .cpp`	Decides which WiFi pop-up notifications to send: a pollution or watch alert starting, a new water speed, and sensor problems (only once the sensors are armed). Remembers "armed" when the board is turned off (`saved_settings.cpp`).
`wifi_alerts.h / .cpp`	Joins WiFi and posts each notification to ntfy.sh in a background task, so a slow connection never holds up the readings. Notifications wait in a queue until WiFi connects.
`secrets.h`	Your WiFi name, password and ntfy topic. You make it by copying `secrets_example.h`. It's in `.gitignore`, so it never goes onto GitHub.
`bluetooth.h / .cpp`	The Bluetooth Low Energy link to the phone. The board shows up as `RiverNode-N1` and sends each message in pieces small enough for one Bluetooth packet.

Testing without hardware

`sim.h / .cpp`	When `SIMULATE` is 1, it makes up readings that follow a 45-minute script: normal water, a rain storm, normal water again, then an acid spill. It also runs fake neighbour nodes (upstream, other bank, downstream) through the same engine, so the network comparison has data.
`test/`	71 unit tests that run on a PC (`pio test -e native`): the rules, the conversions, the fault checks, the baseline, persistence, the whole engine on the simulator script, the river network including branches, the water speed, and the phone messages and notifications.

SECTION 4

Turning voltages into readings

The ESP32 reads voltages on its ADC1 pins (ADC2 can't be used while WiFi is on). The pH module and the TS-300B turbidity sensor are 5 V parts, so each one is wired through a voltage divider: **sensor output → 12k → ESP32 pin → 10k → GND**. The pin sees 10/22 of the sensor voltage (the turbidity sensor's 4.5 V maximum

arrives as 2.05 V), and the code multiplies back by `22 / 10 = 2.2` before converting. Each reading is the average of 16 quick ADC reads to smooth out noise.

READING	PIN	HOW IT'S WORKED OUT (`COMPENSATION.CPP`)
Water temperature	`GPIO 4`	The DS18B20 sends a number directly over its one-wire bus. It gives –127 °C if unplugged and 85 °C before its first reading, so `temperature_valid()` rejects both. The node asks for a new reading each tick and collects it on the next tick, so it never waits.
pH	`GPIO 34`	A straight line through two calibration points: the voltage in pH 7 buffer (`PH_V7` = 2.50 V) and in pH 4 buffer (`PH_V4` = 3.05 V). `pH = 7 + (V − PH_V7) × 3 / (PH_V7 − PH_V4)`
TDS	`GPIO 33`	The probe's curve gives electrical conductivity: `EC = 133.42V³ − 255.86V² + 857.39V` (µS/cm). Warm water conducts better, so it's corrected to 25 °C: `EC25 = EC / (1 + 0.02 × (T − 25))` . Then `TDS = EC25 × 0.5` mg/L. If the temperature sensor fails, 25 °C is used.
Turbidity	`GPIO 32`	The voltage is first scaled to your sensor's clear-water voltage (`TURB_V_CLEAR`), then the curve gives NTU: `−1120.4V² + 5742.3V − 4352.9` , kept between 0 and 3000. Clearer water gives a higher voltage. The TS-300B is rated 0–1000 NTU, so readings above that are only rough. The curve is very steep near clear water (0.1 V below the clear-water voltage already reads about 330 NTU), so measure `TURB_V_CLEAR` for your own sensor.

> **Calibrating**
> Type `cal` in the serial monitor (with `SIMULATE` set to 0). It prints the live sensor voltages. Put the probe in pH 7 buffer and copy the pH voltage into `PH_V7` , do the same in pH 4 buffer for `PH_V4` , and copy the turbidity voltage in clear water into `TURB_V_CLEAR` . `TDS_K_VALUE` can be adjusted using a solution of known TDS.

SECTION 5

Learning what "normal" is

Every river is different, so instead of fixed limits the node learns a **baseline** for each sensor at its own spot (`baseline.cpp`). The rules compare this minute's readings with it.

How it learns

1. Each minute's medians are collected into **slots** of 5 minutes. When a slot fills, its median is saved.
2. The last **288 slots** (24 hours) are kept in a ring buffer: the newest slot overwrites the oldest.
3. The baseline for a sensor is the **median of its slots**. Because it covers a whole day, the normal day–night ups and downs are averaged in.
4. Until the node has **60 minutes** of data it shows `BASELINE_BUILDING` and raises no alerts.

Keeping pollution out of "normal"

If the node kept learning during a spill, the spill would slowly become "normal" and the alert would fade even though the river is still polluted. So the baseline is **frozen**:

- while any rule's pattern is being seen, even if its alert hasn't turned on yet;

- while any label is on (including rain);
- for **3 hours** after the last pattern or label ends (`BASELINE_HOLD_MIN`). This stops pollution that comes and goes from leaking into normal during the quiet gaps;
- for any sensor with a fault. Its readings are never learned.

Extra things it tracks

- **Day pH range:** the lowest and highest pH in each hour of the last 24 hours. The nutrients rule uses it to spot the big day–night pH swing that algae cause. It needs 12 hours of data first. Minutes with a pollution pattern (and the warm-up, before the rules run) are left out, so an acid or alkaline spill can't look like an algae swing.
- **pH probe drift:** every 6 hours a copy of the baseline is saved (12 copies = 3 days). If the pH baseline has moved steadily in one direction by more than 0.5 while the other sensors stayed within 10%, the probe is probably drifting, not the river. The node reports a `PH_RECALIBRATE` fault.

> **Demo mode**
> With `DEMO_MODE` set to 1 (the default) everything is shortened so it can be tested in minutes: 1-minute slots, a 10-minute baseline, a 5-minute warm-up and a 10-minute hold. The nutrients rule and drift check are off in demo mode, because they need many hours. Set it to 0 for real use.

SECTION 6

Sensor fault checks

Before any rule runs, `check_faults()` in `rules.cpp` checks each sensor. A sensor with a fault is left out of the rules and out of the baseline, so a broken probe can't cause a false pollution alert. The node shows FAULT and names the sensor.

FAULT	WHEN	USUALLY MEANS
`NO_READING`	No valid samples this minute. For temperature: the DS18B20 gave −127 or 85 °C.	Unplugged sensor or broken wire. TDS correction falls back to 25 °C.
`OUT_OF_RANGE`	pH outside 0–14, TDS below 0 or higher than the TDS board can output (over 2.6 V), turbidity above 3000 NTU, temperature outside −5 to 50 °C.	Broken probe, bad wiring or bad calibration.
`FLATLINE`	The reading stays within 0.1% for 60 minutes in a row. Clear water sitting at 0 NTU doesn't count, and neither does very salty water holding the TDS sensor at its top (2.2 V or more): that's salt, not a stuck sensor.	A stuck sensor or a disconnected input that reads a constant voltage.
`JUMP`	Water temperature changes by more than 3 °C in one minute *and comes straight back* (to within 1 °C) the next minute. A jump that stays could be real, like a warm outflow reaching the probe, so it's left to the thermal rule.	A glitch: the thermometer was briefly out of the water or has a loose wire.
`PH_RECALIBRATE`	The pH baseline drifts steadily for 3 days (section 5). Real mode only.	The pH probe needs recalibrating in buffers.

SECTION 7

The pollution rules

Each rule in `rules.cpp` looks for one kind of pollution's "fingerprint": how the four readings change compared with normal. Changes are measured three ways:

- **pH and temperature:** the difference (e.g. pH dropped 1.2).
- **TDS:** the percentage rise (e.g. +50% means 1.5 times normal).
- **Turbidity:** how many times normal (e.g. ×3), *and* by how much. Near clear water the TS-300B curve is very steep (4.2 V is about 1 NTU but 4.1 V is about 360 NTU), so small voltage wobbles look like tens of NTU. So "normal" is taken as at least 20 NTU (`BASE_FLOOR_NTU`), and a rise only counts if it's also more than 25 NTU above normal (`NTU_MIN_RISE`). Leave the sensor in still tap water for 30 minutes and see how much it wobbles: set both a little above that.

How confidence is worked out

Every rule has **required** conditions (all must be true or the rule doesn't fire) and some **optional** ones that add confidence. A few rules have a **strong sign** that adds a bonus.

```
confidence = conditions met ÷ total conditions + 0.1 × strong signs (capped at 1.0)
```

Example: heavy metals has 2 required and 1 optional condition. With both required met but clear water: 2 ÷ 3 = 0.67. If the pH is also below 5 (strong sign): 0.67 + 0.1 = 0.77.

Rain filter (checked first)

Rain dilutes the dissolved salts (TDS **down** more than 15%), washes mud in (turbidity **more than 2×** and +25 NTU) and cools the water (**more than 0.5 °C** colder). If all three happen, the industrial, alkaline and sediment rules are skipped, because rain explains those readings. The node stays NORMAL and shows the rain label. The sewage and heavy metals rules still run, because sewage overflows happen mostly during rain and storms wash mine drainage out too; anything they find is shown alongside the rain label. (With only four sensors, a sewage overflow mixed with rain often looks like rain. The network finding "rain at this node only" is the better clue to an overflow or outfall.)

The rules and their numbers

LABEL	REQUIRED (ALL OF THEM)	OPTIONAL / BONUS
ALERT **Heavy metals / acid mine drainage**	pH below 6.0 *or* dropped more than 1.0 TDS more than 50% above normal	Turbidity more than 1.5× (and +25 NTU) Bonus: pH below 5.0
ALERT **Industrial discharge**	pH below 6.0 *or* above 9.0 A sudden TDS jump: more than 30% above its value up to 30 min ago, and more than 30% above normal	Temperature more than 2 °C above normal
ALERT **Alkaline waste**	pH above 9.0 TDS more than 20% above normal	Turbidity more than 1.5× (and +25 NTU) Bonus: pH above 10.0
ALERT **Sewage**	TDS more than 20% above normal (no upper limit, so a big spill into a low-salt river still counts) Turbidity more than 2× and +25 NTU	pH dropped 0.3–1.0 Temperature up 0.5–2 °C

WATCH **Nutrients / algae**	12 h of pH history, and the day's pH swing is more than 1.0 *or* pH above 9.0	TDS 10–30% above normal (fertiliser nitrate is only a few mg/L, so this often won't move) Water warmer than 25 °C
WATCH **Thermal pollution**	Temperature more than 3 °C above normal *or* above 30 °C (shallow streams warm 2–4 °C on a sunny afternoon, and "normal" is a 24 h median, not the time of day) Everything else normal: pH within 0.3, TDS within 10%, turbidity not up	–
WATCH **Sediment**	Turbidity more than 3× *or* above 50 NTU, and more than 25 NTU above normal TDS within 10% of normal pH within 0.3 of normal	–
WATCH **Salt / saline water**	TDS above 900 mg/L (the TDS sensor tops out at about 1120 mg/L) pH between 0.3 below and 1.0 above normal (seawater is about pH 8.1) Turbidity not up	–
WATCH **Treated wastewater or fertiliser runoff**	TDS more than 15% above normal pH dropped 0.25–1.0 (a bit above cheap-probe drift) Turbidity not up (muddy water is the sewage rule)	Temperature up 0.3–3 °C

Several rules can fire in the same minute. For example, strong acid with a sudden TDS jump matches both heavy metals and industrial discharge. They're all kept, and the next step decides which one is shown as the main label.

> **Why these fingerprints?**
> Acid mine water is very acidic and full of dissolved metals (low pH, high TDS). Factory leaks start suddenly at one point (the TDS jump). Cement and concrete washwater is strongly alkaline. Raw sewage is cloudy and adds salts. Treated sewage and fertiliser add nitrate and salts without mud, and slightly lower the pH. Algae fed by nutrients push pH up by day and down at night. Mud from land clearing is cloudy but doesn't change the chemistry. Warm outflows change only the temperature. Seawater or brine is extremely salty but otherwise clear.

SECTION 8

From rules to alerts

Persistence: no flickering

A single odd minute shouldn't raise an alarm. `persistence.cpp` keeps a streak counter for every label:

- A label turns **on** after its rule fires **3 minutes in a row** (`PERSIST_ON_MIN`).
- It turns **off** after the rule hasn't fired for **5 minutes in a row** (`PERSIST_OFF_MIN`).
- Each change prints a line straight away: `>>> ALERT ON: Likely sewage (conf 0.75)` or `>>> ALERT OFF: Likely sewage` .

Choosing the node's state and main label

`build_report()` in `node_engine.cpp` picks one state, in this order of importance:

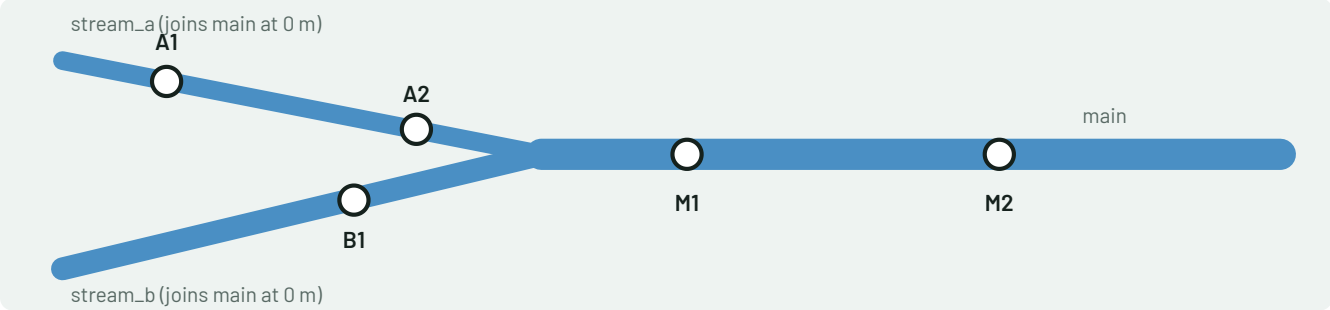
1. Any **pollution** label on → ALERT. The main label is the one with the highest confidence.
2. Otherwise, any **sensor fault** → FAULT.
3. Otherwise, any **watch** label on → WATCH.
4. Otherwise → NORMAL (showing the rain label if it's raining).

Every other label that's on is listed as *"Also flagged"*, so nothing is hidden. Pollution beats a fault because a fault only knocks out one sensor, and the rules for the working sensors may still have found something real.

SECTION 9

Comparing nodes along the river

`network.cpp` keeps a list of every node: its id, place name, **branch** (which stream or river it's on), distance down that branch, and latest report. The list is kept in order from upstream to downstream. Nodes on the same branch at the same distance are at the same **position**, e.g. the left and right bank at one spot.



The nearest upstream nodes of M1 are A2 (on stream A) *and* B1 (on stream B). A1 isn't one of them, because A2 is closer on that stream.

Each minute, `network_assess()` looks at the other nodes that have reported in the last 3 minutes (older ones count as offline). It finds the **nearest upstream node on every stream** above this one and the nodes at the same position, then picks a finding:

FINDING	WHEN	WHAT IT TELLS YOU
`FROM_UPSTREAM`	This node is on alert and a nearest upstream node has the same alert.	The pollution started further up. Shows the river distance between them.
`SOURCE_CROSS_SECTION`	On alert, and every other node at this position is too.	The whole width of the river is affected here.
`SOURCE_LOCAL_SIDE`	On alert, but other nodes at this position are normal.	The source is on this bank, between the upstream node and here.
`SOURCE_BETWEEN`	On alert, the nearest upstream node(s) are normal.	The source is between them and this node.
`NO_UPSTREAM`	On alert, other nodes are working but none are upstream.	The source is somewhere above this node.
`NO_PEERS`	On alert or rain, but no other node has fresh data.	Nothing to compare with.
`RAIN_CONFIRMED`	Rain pattern here and at more than half of all nodes.	A real weather event.
`RAIN_UNCONFIRMED`	Rain pattern here but not at most other nodes.	Possibly a stormwater pipe or a leak nearby that only looks like rain.

Branches are set up with `network_add_branch(name, length, flows_into, joins_at)`. The river a stream flows into must be added first. `network_add_node()` on its own puts a node on a branch called `main`, so a single straight river needs no branch setup. `network_river_distance()` works out how far the water travels from one node to another, following it down through any joins.

> **Important: the radio link isn't written yet**
> The network comparison works, but each board currently only knows its own report. The reports from the other nodes in `setup_river_layout()` come from the simulator (`SIM_REMOTE_NODES`). To connect real boards, a LoRa or WiFi link would receive each neighbour's `Node_report` and pass it to `network_update_report()` , which is already the entry point for it.

SECTION 10

Reading the serial output

Every minute the node prints a block like this (115200 baud). This one is from the built-in simulator during its acid spill:

```
>>> ALERT ON: Likely heavy metals (conf 1.00)
================ NODE N1 | 00:04:20 | ALERT =================
Reading     Now       Normal    Change
pH          4.60      7.20      -2.61
TDS         617 mg/L  210 mg/L  +194%
Turbidity   18.2 NTU  8.2 NTU   x2.2
Water temp  17.4 C    17.1 C    +0.3 C
------------------------------------------------------------
[POLLUTION] Likely heavy metals / acid mine drainage  (conf 1.00)
  Possible pollutants: Iron, aluminium, manganese, copper, zinc, lead,
                       cadmium, arsenic, sulfate / sulfuric acid
  What it means:       Acidic, metal-rich water. Low pH also pulls
                       metals out of the riverbed.
  Also flagged:        LIKELY_INDUSTRIAL_DISCHARGE
  Note: Pollutants listed are likely candidates only.
        Confirm with a lab water sample.
============================================================
Network (upstream -> downstream):
  N0        0m  upstream     NORMAL
  N1     1000m  left bank    ALERT LIKELY_HEAVY_METALS  <- this node
  N2     1000m  right bank   NORMAL  <- same position
  N3     2500m  downstream   ALERT LIKELY_HEAVY_METALS
  >> Source likely between N0 (0 m) and N1 (1000 m), on N1's
     side - other node(s) at this position are normal.
JSON:{"node":"N1","pos":1000,"t":260,"state":"ALERT","label":"LIKELY_HEAVY_METALS",
"conf":1.00,"also":["LIKELY_INDUSTRIAL_DISCHARGE"],"ph":4.60,"tds":617,"ntu":18.2,...}
```

1. **The header** shows the node, how long it has been running, and its state, in the state's colour.
2. **The table** compares this minute with normal, using the same change measures as the rules.
3. **The label** gives its group, name, confidence, likely pollutants, meaning, and any other labels that are on.
4. **The network list** shows every node's state, and the `>>` line is the network finding from section 9. Here N0 above is normal and N2 on the other bank is normal, so the source is on N1's side between 0 and 1000 m.
5. **The JSON line** has the same information for a computer, e.g. a logging script or a dashboard. `OUTPUT_MODE` chooses human text, JSON, or both.

SECTION 11

Water speed and phone alerts

Water speed

There's no flow sensor, so the speed is worked out from pollution moving down the river (`water_speed.cpp`). When the same kind of pollution turns on at an upstream node and later at a node further down:

```
N0 alert at 10:00  ---- 1500 m of river ---->  N1 alert at 10:25
speed = 1500 m / (25 x 60 s) = 1.0 m/s
```

To avoid made-up speeds, a pair only counts when:

- both are **fresh arrivals**: the node had no pollution or watch alerts for the hour before (`SPEED_QUIET_MIN`)

- it's the **same label**, and the upstream arrival was in the last 6 hours (`SPEED_PAIR_WINDOW_MIN`)
- there was **no rain** in between, because rain pauses the rules and the alerts restart afterwards
- the second node really is **downstream** (the distance follows the water, through stream joins), and the speed is under 5 m/s (`SPEED_MAX_MPS`)

The closest upstream arrival is used. A new speed prints a `>>> WATER SPEED` line and is sent to the phone.

Pop-up notifications over WiFi

This is the main way to get alerts. The board joins WiFi and sends notifications through **ntfy**, a free app for iPhone and Android (`wifi_alerts.cpp`, `phone_alerts.cpp`). They pop up like a text message, even on the lock screen:

NOTIFICATION	WHEN	WHAT'S IN IT
`POLLUTION ALERT / Watch`	A pollution or watch label turns on (urgent for pollution)	The label(s) and how sure, the 4 readings next to normal, likely pollutants, where the source probably is, and the water speed if known
`Water speed`	A new water speed is worked out	Which two nodes, how far apart, how long it took, m/s and km/h
`Sensor problem`	A sensor fault lasts 3 minutes (`FAULT_NOTIFY_MIN`), only once the sensors are **armed**	Which sensor and what's wrong. Sent once, and again only if it's fixed and breaks again
`Online / Test / Armed`	The board turns on, `test` is typed, or the sensors are armed or disarmed	The current readings. Arming also lists any sensor that has a problem right now

Arming: while the probes are being set up, unplugging them would set off sensor-problem notifications. So these only start once you type `arm` in the serial monitor (or tap **Arm sensors** on the Bluetooth page). It's remembered when the board is turned off. `disarm` turns them off again.

Setting up: copy `src/secrets_example.h` to `src/secrets.h` and fill in the WiFi name, password and your ntfy topic, then subscribe to the same topic in the ntfy app. The ESP32 only joins 2.4 GHz WiFi without a login page; a phone hotspot works (on an iPhone, turn on "Maximise Compatibility"). `wifi` in the serial monitor shows whether it's connected and how many notifications were sent. Anyone can open `ntfy.sh/<your topic>` in a browser to watch the alerts arrive without installing anything.

Phone page over Bluetooth (backup)

The board sends everything to a web page on the phone over Bluetooth Low Energy (`bluetooth.cpp`), using the Nordic UART service, a common standard for sending text. Each message is one line of JSON (`phone_messages.cpp`):

MESSAGE	WHEN	WHAT'S IN IT
`alert`	As soon as a label turns on or off	The label, its likely pollutants, and the 4 readings at that moment
`status`	Every minute, and when the phone connects	The 4 readings and their normal levels, state, main label, confidence, pollutants, also-flagged labels, sensor faults, the network finding and the latest water speed
`nodes`	Every minute	Every node on the river, its state and readings

speed	When a new water speed is found	The speed, which two nodes, how far apart and how long it took

The phone page is `docs/phone/index.html`, published by GitHub Pages at `barkshavings008.github.io/River-monitoring-nodes/phone/`. It shows the readings, what the node thinks is in the water, the water speed and the other nodes. When an alert starts it pops up a banner and beeps. The page can also ask the board to resend everything ("Refresh now"), and it reconnects by itself if the board goes out of range for a moment.

> **On an iPhone**
> Safari can't use Bluetooth, so open the page in the free **Bluefy** browser app. Keep the page open with the screen on: iPhones pause web pages in the background. The page has a **Try a demo** button that replays real messages from the board's simulator, so it can be tried without the board.

SECTION 12

Settings, building and uploading

The switches in `config.h`

SETTING	DEFAULT	WHAT IT DOES	FOR REAL USE
`SIMULATE`	1	1 = scripted fake readings, 0 = read the real sensors. Also makes a "minute" last 10 s (`SIM_FAST`).	0
`SIM_REMOTE_NODES`	1	Fakes the other nodes on the river so the network comparison has data.	0 once a radio link exists
`DEMO_MODE`	1	Short learning times for quick testing (section 5).	0
`OUTPUT_MODE`	BOTH	Human text, JSON lines, or both.	Your choice
`USE_COLOUR`	1	Coloured text. Turn off if your serial monitor shows codes like `[31m`.	Your choice
`NODE_ID`	"N1"	This board's id in the network.	Unique per board
`WIFI_ALERTS`	1	Pop-up notifications through ntfy over WiFi, once `src/secrets.h` is filled in (section 11).	1
`BLUETOOTH`	1	Sends everything to the phone page over Bluetooth (section 11).	1 (0 saves power)

Uploading to the board

1. Open the folder that contains `platformio.ini` in VS Code with the PlatformIO extension (File > Open Folder).
2. Set the switches above in `src/config.h`. For WiFi notifications, copy `src/secrets_example.h` to `src/secrets.h` and fill it in.
3. Plug in the ESP32 by USB and click the → **Upload** arrow in the PlatformIO bar at the bottom. PlatformIO compiles every file in `src/` together, not just `main.cpp`, and downloads the OneWire, DallasTemperature and NimBLE (Bluetooth) libraries by itself.

4. Click the **plug icon (Serial Monitor)** to watch the output and type commands.

> **Board type**
> `platformio.ini` is set for a classic ESP32 DevKit (`board = esp32dev`), and the pins in `config.h` (34, 33, 32, 4) are ESP32 DevKit pins. An ESP32-C3 has different pins and needs the board and pins changed first.

Running the tests on a computer

`pio test -e native` builds the logic files (not the hardware ones) for your computer and runs all 71 tests. Run it after changing a rule or a threshold, to check nothing else broke.

SECTION 13

The other folders

`manual/`	A separate, simpler PlatformIO project for writing the logic yourself. It only has helpers: • `return_ph()` , `return_tds()` , `return_turbidity()` and `return_water_temp()` , with temperature correction built in • a linked list of river nodes with add, delete, move, find, distance and print functions • a `FAKE_SENSORS` switch for testing at a desk Its `main.cpp` is a short example. See `manual/README.md` .
`simulator/`	An interactive web page with two streams joining a river, six nodes and three factories whose waste you can choose. It runs **the real code from `src/`** , compiled to WebAssembly: `simulator/build.sh` rebuilds it after the code changes. The page works out the water at each node and turns it into sensor voltages. Everything after that is the node code. Open `simulator/index.html` in a browser.
`test/`	The PC unit tests (section 12), in five groups: `test_rules` , `test_engine` , `test_network` , `test_speed` and `test_phone` .
`platformio.ini`	Build settings: the `esp32dev` board for uploading, and a `native` environment that runs the tests on a computer.
`include/, lib/`	Empty PlatformIO folders, kept for later.

SECTION 14

Limits and things to know

- **Labels are likely, not certain.** Different pollution can look alike to four sensors. Strong acid often matches both heavy metals and industrial discharge, and the front edge of a diluted acid plume can briefly match the effluent / fertiliser rule. The node shows every match and lists the likely pollutants so a water sample can be tested.
- **The TDS probe tops out.** The STJF board outputs at most about 2.3 V, roughly 1120 mg/L at 25 °C. The salt rule fires above 900 mg/L, and a reading pinned at the top counts as salt, not a stuck sensor. But the node can't tell brackish water from seawater: both just read "at the top".

- **Turbidity limits are a trade-off.** The 20 NTU floor and +25 NTU rise stop wobble on the steep part of the TS-300B curve causing false alerts, but they also mean a diluted sewage overflow is usually only caught at the node nearest the outfall. Set them from your own tap-water wobble test.
- **Slow-to-show pollution.** The nutrients rule needs 12 hours of pH history and isn't used in demo mode. Diluted pollution far downstream may fall just under a rule's threshold.
- **A permanent change stays an alert.** Because "normal" is frozen while patterns are seen and for 3 hours after, a real lasting change in the river keeps the node on alert. Restarting or moving the node makes it relearn (1 hour).
- **No real radio link yet.** The network comparison is ready, but neighbour reports currently come from the simulator (section 9).
- **WiFi notifications need WiFi and internet.** Rivers rarely have WiFi, so a real deployment would use LoRa radio or a mobile (cellular) connection. The Bluetooth page is the backup, but it only beeps while it's open with the screen on (iPhones pause web pages in the background), and Bluetooth only reaches about 10–30 m.
- **Water speed needs a passing plume.** It's only known after the same pollution reaches two nodes, and it's the speed of the pollution, which is close to the water's speed but can lag behind it a little.
- **No clock.** Times are counted from when the board turned on. The nutrients rule treats "daytime pH above 9" as any pH above 9 for this reason.

SECTION 15

Words used in this guide

ADC	Analog-to-digital converter: the part of the ESP32 that measures a voltage.
Baseline	The node's learned "normal" for each sensor: the median of the last 24 hours of clean readings.
Branch	One stream or river in the network. A branch can flow into another.
Confidence	How many of a rule's conditions were met, from 0 to 1 (section 7).
EC	Electrical conductivity: how well water carries electricity. More dissolved salts means higher EC.
Median	The middle value once a list is sorted. Unlike an average, one wild value doesn't move it.
Node	One monitoring station: an ESP32 with the four sensors.
NTU	Nephelometric turbidity units: how cloudy water is. Drinking water is under 1, a muddy river can be hundreds.
Persistence	Requiring a pattern for 3 minutes before alerting and 5 minutes without it before clearing.
pH	Acidity on a 0–14 scale. 7 is neutral, lower is acidic, higher is alkaline. Each step is ten times stronger.
Position	A spot on the river (branch + distance). Several nodes can share one, e.g. both banks.
TDS	Total dissolved solids, in mg/L (milligrams per litre): salts and minerals dissolved in the water.

Upstream / downstream	Towards where the water comes from / towards where it goes.